

DBMS Unit 2

Entity Relationship Modelling and Constraints in Database

RV University | 2nd Semester Computer Science | CIE 1 Exam-Ready Reference

1. ER Model — Foundational Concepts

1.1 What is an ER Model?

The **Entity-Relationship (ER) Model** is a high-level conceptual data model that graphically represents the logical structure of a database. Introduced by **Peter Pin-Shan Chen in 1976**, it is the standard tool used during the **database design phase** to describe data and its relationships before implementation in a DBMS.

An **ER Diagram (ERD)** is the visual representation of the ER Model. It acts as a blueprint that maps real-world domains to a database schema.

Purpose: Translate real-world information requirements into a formal, implementation-independent schema that can then be mapped to a relational, object-oriented, or other database model.

Position in Database Design Workflow:

Requirements Analysis

↓

Conceptual Design (ER Model ← YOU ARE HERE)

↓

Logical Design (Relational Schema)

↓

Physical Design (Storage, Indexing)

1.2 Entity

Definition: An **entity** is any real-world object or concept that has an independent existence and can be distinctly identified. It is the basic building block of an ER Model.

Types of Entities

Type	Definition	ER Symbol	Example
Strong Entity	Has a primary key; can be uniquely identified without depending on another entity	Rectangle (single border)	STUDENT, EMPLOYEE
Weak Entity	Cannot be uniquely identified by its own attributes alone; depends on an owner/identifying entity	Rectangle (double border)	DEPENDENT (depends on EMPLOYEE), ORDER_ITEM (depends on ORDER)

Key distinction for Weak Entities:

- Weak entities use a **partial key** (also called a **discriminator**), represented by a dashed underline.
- The relationship connecting a weak entity to its owner is called an **identifying relationship**, drawn as a **double diamond**.
- The **full key** of a weak entity = partial key + primary key of the owner entity.

Example: Consider EMPLOYEE (strong) and DEPENDENT (weak). A dependent named "Alice" cannot be identified without knowing which employee she depends on. Her identity = (emp_id, dep_name).

Entity Set vs. Entity Instance

Term	Meaning
Entity Instance	A single real-world object (e.g., the student "Rahul Sharma" with ID 101)
Entity Set	The collection of all entities of the same type at a given point in time (e.g., all students enrolled) — this is what maps to a table in relational schema

1.3 Attributes

Definition: An **attribute** is a property or characteristic that describes an entity or a relationship. Attributes represent the data we store about entities.

Classification of Attributes

A. Simple (Atomic) Attribute

- **Cannot be subdivided** into smaller meaningful components.
- Stored as a single atomic value.
- **Example:** Age, Employee_ID, Salary

B. Composite Attribute

- **Can be decomposed** into sub-attributes, each with independent meaning.
- **Example:** Name → {First_Name, Middle_Name, Last_Name}; Address → {Street, City, State, Zip}
- In ER notation: sub-attributes branch off the composite attribute oval.
- **Reduction rule:** Each component becomes a separate column in the table.

C. Single-Valued Attribute

- Holds exactly **one value** per entity instance.
- **Example:** Date_of_Birth, Employee_ID

D. Multivalued Attribute

- Can hold **multiple values** for a single entity instance.
- Represented in ER diagram by a **double oval**.
- **Example:** Phone_Numbers (a person may have home, work, mobile), Degrees_Held
- **Reduction rule:** Creates a **separate table** in relational schema.

E. Derived Attribute

- Value can be **computed** from other stored attributes; not physically stored.
- Represented by a **dashed oval**.
- **Example:** Age (derived from Date_of_Birth and current date), Total_Salary (derived from Base + Allowances)

F. Key Attribute

- Uniquely identifies each entity in the entity set.
- Represented by an oval with the attribute name **underlined**.
- **Example:** Student_ID in STUDENT, SSN in EMPLOYEE

G. NULL Attribute

- An attribute that may hold a **NULL value** — meaning the value is unknown, not applicable, or missing.

- **Example:** `Middle_Name` can be NULL if a person has no middle name.

Attribute Type Summary Table

Attribute Type	Description	ER Notation	Table Mapping
Simple	Atomic, indivisible	Single oval	Single column
Composite	Decomposable	Oval with sub-ovals	Multiple columns (components)
Single-valued	One value per entity	Single oval	Single column
Multivalued	Many values per entity	Double oval	Separate table
Derived	Computed from others	Dashed oval	Usually not stored (or stored with note)
Key	Unique identifier	Oval + underlined name	Primary key column

Instance: Entity STUDENT with attributes:

- `Student_ID` → Key (simple, single-valued)
- `Name` → Composite (`First_Name`, `Last_Name`)
- `DOB` → Simple, single-valued
- `Phone` → Multivalued (double oval)
- `Age` → Derived from `DOB`

1.4 Relationship

Definition: A **relationship** is a logical association or interaction between two or more entities. It represents how entity instances are connected.

Relationship Set vs. Relationship Instance

Term	Meaning
Relationship Instance	A single association between specific entity instances (e.g., "Rahul enrolled in CS101")
Relationship Set	The set of all relationship instances of the same type (e.g., all ENROLLED_IN associations between students and courses)

Degree of a Relationship

Degree	Name	Description	Example
1	Unary (Recursive)	Entity relates to itself	EMPLOYEE manages EMPLOYEE
2	Binary	Two entity sets	STUDENT enrolled_in COURSE
3	Ternary	Three entity sets	SUPPLIER supplies PART to PROJECT
n	n-ary	n entity sets	Generalization of above

Note: Most real-world relationships are **binary**. Ternary and higher relationships are reduced to binary when possible.

Relationship Attributes

Relationships themselves can have **descriptive attributes** — attributes that describe properties of the relationship, not of either participating entity alone.

Example: The relationship STUDENT enrolled_in COURSE may have attributes Grade and Enrollment_Year, because these describe the enrollment, not the student or course individually.

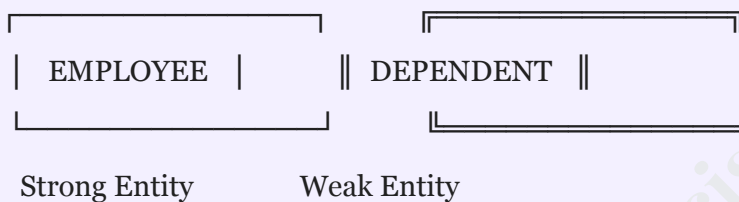
2. ER Diagram Notation

2.1 Notation Summary Table

Concept	Symbol
Strong Entity	Rectangle (single border)
Weak Entity	Rectangle (double border)
Simple Attribute	Oval
Composite Attribute	Oval with branching sub-ovals
Multivalued Attribute	Double oval
Derived Attribute	Dashed oval
Key Attribute	Oval with underlined attribute name

Relationship (Strong)	Diamond (single border)
Identifying Relationship (Weak)	Diamond (double border)
Participation (Total)	Double line connecting entity to relationship
Participation (Partial)	Single line connecting entity to relationship
Cardinality (1)	"1" near the entity
Cardinality (Many)	"N" or "M" near the entity

2.2 Entity Notation



→ ER Notation Reference Diagram

2.3 Attribute Notation

Name (Composite)

/ | \

First Middle Last

(Phone) ← Multivalued (drawn as double oval)

(Age) ← Derived (drawn as dashed oval)

EmpID ← Key Attribute (underlined)

→ Visual ER Attribute Types

2.4 Relationship Notation

EMPLOYEE ———— ◇ WORKS_FOR ———— DEPARTMENT

Manages (binary relationship)

EMPLOYEE ════════◇ HAS ════════ DEPENDENT

Identifying relationship (double diamond/double line)

→ Full ER Diagram Notation Guide

3. Mapping Cardinality Constraints

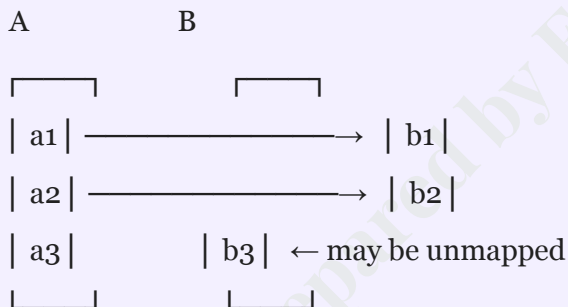
Definition: Mapping cardinality (or **cardinality ratio**) expresses the number of entity instances in one entity set that can be associated with the number of entity instances in another entity set through a given relationship set.

Most directly applicable and useful for **binary relationship sets**.

The four possible cardinality ratios for a binary relationship between entity sets **A** and **B** are:

3.1 One-to-One (1:1)

Formal definition: Each entity in A is associated with **at most one** entity in B, and each entity in B is associated with **at most one** entity in A.



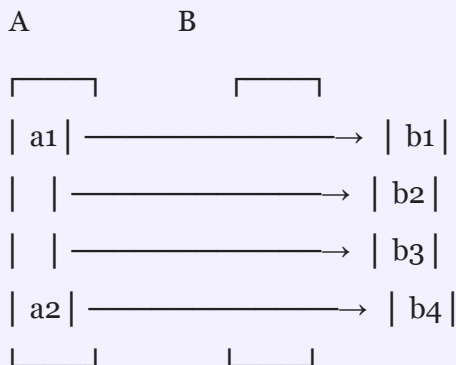
Example: PERSON **has** PASSPORT — one person holds at most one passport; one passport belongs to at most one person.

Example: EMPLOYEE **manages** DEPARTMENT (if each dept has exactly one manager and each manager manages exactly one dept).

SQL Implication: Foreign key placed in either table; typically placed in the table with fewer NULLs.

3.2 One-to-Many (1:N)

Formal definition: One entity in A can be associated with **many** (zero or more) entities in B, but each entity in B is associated with **at most one** entity in A.



Example: DEPARTMENT **employs** EMPLOYEE — one department can have many employees, but each employee belongs to one department.

Example: CUSTOMER **places** ORDER — one customer can place many orders, each order belongs to one customer.

SQL Implication: Foreign key placed in the "**many**" side (B's table) referencing A's primary key.

3.3 Many-to-One (N:1)

Formal definition: Many entities in A can be associated with **at most one** entity in B, but each entity in B can be associated with **many** entities in A.

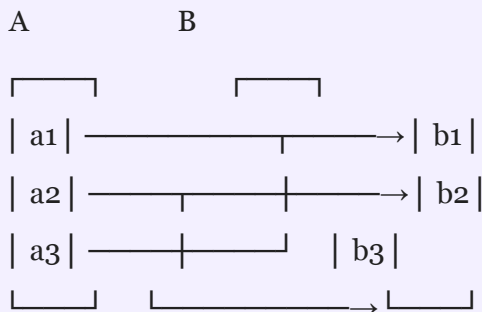
This is simply the **mirror** of 1:N — the distinction depends on which entity set you label "A" and "B."

Example: EMPLOYEE **works_for** DEPARTMENT — many employees work for one department.

SQL Implication: Same as 1:N — foreign key placed on the "many" side.

3.4 Many-to-Many (M:N)

Formal definition: Each entity in A can be associated with **many** entities in B, and each entity in B can be associated with **many** entities in A.



Example: STUDENT **enrolled_in** COURSE — one student takes many courses; one course has many students enrolled.

Example: ACTOR **appears_in** MOVIE.

SQL Implication: Requires a **junction/bridge table** (relationship table) with foreign keys referencing both entity tables.

3.5 Participation Constraints

Participation constraints specify whether the **existence** of an entity depends on its participation in a relationship.

Type	Definition	ER Notation	Also Called
Total Participation	Every entity in the set must participate in at least one relationship instance	Double line between entity and relationship	Existence dependency
Partial Participation	Some entities may not participate in any relationship instance	Single line between entity and relationship	Optional participation

Example: In EMPLOYEE works_for DEPARTMENT:

- **EMPLOYEE** has **total participation** (every employee must work for some department) → double line
- **DEPARTMENT** has **partial participation** (a department might exist with no employees yet) → single line

→ **Participation Constraints Visual**

4. Integrity Constraints

4.1 Motivation and Role

Definition (formal): An **integrity constraint (IC)** is a condition specified on a database schema that restricts the data that can be legally stored in any instance of that database.

Why ICs are necessary:

- Prevent **semantically incorrect data** from entering the database (e.g., a student's age = -5).
- Enforce **business rules** at the data layer.
- Eliminate the need to duplicate validation logic across multiple application programs.

How the DBMS enforces ICs:

- Only **legal database instances** (those satisfying all constraints) are permitted.
- The DBMS automatically checks all constraints on every INSERT, UPDATE, and DELETE operation.

Constraint Specified at Schema Definition Time



DBMS Stores Constraint Metadata



On Every DML Operation (INSERT / UPDATE / DELETE)



DBMS Checks All Applicable Constraints



All Satisfied	Any Violated	
→ Commit	→ Rollback	

Categories of Integrity Constraints:

1. Domain Constraints
 2. Key Constraints
 3. Entity Integrity Constraint
 4. Referential Integrity Constraint
-

4.2 Domain Constraints

Definition: A **domain constraint** specifies the set of permissible values for an attribute. Every attribute value in every tuple must be an element of the attribute's domain (data type + valid value range).

Components of a domain definition:

- **Data type:** `INTEGER`, `VARCHAR(n)`, `DATE`, `FLOAT`, `BOOLEAN`, etc.
- **Range restriction:** `CHECK (age BETWEEN 0 AND 150)`
- **Format restriction:** `CHECK (phone_number ~ '^[0-9]{10}$')`
- **Nullability:** `NOT NULL` or `NULL`

SQL Syntax:

```
CREATE TABLE EMPLOYEE (  
    emp_id    INTEGER    NOT NULL,  
    name      VARCHAR(100) NOT NULL,  
    age       INTEGER    CHECK (age >= 18 AND age <= 65),  
    salary    DECIMAL(10,2) CHECK (salary > 0),  
    gender    CHAR(1)    CHECK (gender IN ('M', 'F', 'O'))  
);
```

Instance: If you attempt to INSERT an employee with age = -3, the DBMS rejects the operation — domain constraint violation.

4.3 Entity Integrity Constraint

Definition: The **entity integrity constraint** states that **no primary key attribute of any tuple in a base relation is allowed to be NULL**.

Formal statement:

If attribute A is a primary key of relation R, then for all tuples t in R: $t[A] \neq \text{NULL}$

Rationale: The primary key is the unique identifier of a tuple. A NULL primary key means the tuple cannot be identified — this defeats the fundamental purpose of the primary key.

Important nuances:

- Applies to **all** attributes of a **composite primary key** (none of the components may be NULL).
- NULL is **not** the same as 0 or an empty string — it means "unknown/inapplicable."

- Unique key constraints allow NULL (with restrictions), but primary keys never allow NULL.

Example: In a STUDENT(student_id, name, dob) table where student_id is the PK, inserting a row with student_id = NULL is a violation of entity integrity.

-- This will always fail if student_id is PRIMARY KEY

INSERT INTO STUDENT VALUES (NULL, 'Rahul', '2004-05-10');

-- ERROR: null value in column "student_id" violates not-null constraint

4.4 Referential Integrity Constraint

Definition: The **referential integrity constraint** ensures that a value appearing in a **foreign key** column in one relation must either appear as a **primary key value in the referenced relation** or be **NULL** (if allowed).

Formal statement:

If relation R2 has a foreign key FK referencing the primary key PK of relation R1, then for every tuple t2 in R2: t2[FK] = NULL **OR** there exists a tuple t1 in R1 such that t2[FK] = t1[PK]

Terminology:

Term	Meaning
Referencing relation	The table that contains the foreign key (child table)
Referenced relation	The table whose primary key is being referenced (parent table)
Dangling reference	A foreign key value that has no matching primary key in the referenced table — a violation

Example: Table ENROLL(student_id, course_id) references STUDENT(student_id). If you try to insert ENROLL(999, 'CS101') but student 999 does not exist in STUDENT, referential integrity is violated.

Violation scenarios and DBMS actions:

Scenario	Default Action	ON DELETE CASCADE	ON DELETE SET NULL
DELETE parent tuple referenced by FK	REJECT	Delete child rows	Set FK to NULL

UPDATE parent PK referenced by FK	REJECT	Cascade update	Set FK to NULL
INSERT child row with non-existent FK	REJECT	N/A	N/A

SQL Example:

```
CREATE TABLE DEPARTMENT (  
  dept_id INTEGER PRIMARY KEY,  
  dept_name VARCHAR(50)  
);
```

```
CREATE TABLE EMPLOYEE (  
  emp_id INTEGER PRIMARY KEY,  
  name VARCHAR(100),  
  dept_id INTEGER,  
  FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE  
);
```

Partial vs. Total Referential Integrity:

- If FK is **NOT NULL**: every child row must have a matching parent row (total referential integrity).
- If FK is **nullable**: a child row can exist without a parent (the NULL FK means "no association").

4.5 Key Constraints

Definition: A **key constraint** specifies that no two distinct tuples in a relation can have the same value for the key attribute(s). Key constraints enforce **uniqueness**.

This is covered in depth in Section 5.

5. Keys in Detail

Keys are the foundation of relational integrity. A **key** is an attribute (or minimal set of attributes) whose value(s) uniquely identify a tuple in a relation.

5.1 Super Key

Definition: A **super key** is any set of one or more attributes whose combined values are guaranteed to uniquely identify each tuple in a relation. A super key may contain redundant attributes.

Formal definition: A set of attributes SK of relation R is a super key of R if, for any two distinct tuples t_1 and t_2 in any valid instance of R: $t_1[SK] \neq t_2[SK]$

Properties of Super Keys:

- **Uniqueness:** Guaranteed for all valid instances.
- **Originality:** Ensures accurate data retrieval.
- **Non-minimal:** May contain unnecessary attributes.
- Every **superset** of a super key is also a super key.
- Every candidate key is a super key, but not every super key is a candidate key.

Example: Relation STUDENT(student_id, name, email, phone)

Super Key	Is it minimal?
{student_id}	Yes (candidate key)
{email}	Yes (candidate key)
{student_id, name}	No (name is redundant)
{student_id, email, phone}	No
{student_id, name, email, phone}	No

All of the above are super keys. Only the first two are candidate keys.

→ **Super Key Visual Explanation**

5.2 Candidate Key

Definition: A **candidate key** is a **minimal super key** — a super key from which no attribute can be removed without losing the uniqueness property.

Formal definition: A set of attributes CK is a candidate key of R if:

1. **CK** is a super key of **R** (uniqueness property), **AND**
2. No proper subset of **CK** is a super key of **R** (minimality property)

Properties:

- A relation can have **one or more** candidate keys.
- All candidate keys are **eligible** to be chosen as the primary key.
- Candidate keys satisfy both **uniqueness** and **minimality**.
- They possess two essential characteristics: **uniqueness** and **minimal redundancy**.

Example: In STUDENT(student_id, name, email, phone):

- Assume student_id and email are both unique.
- {student_id} → candidate key ✓
- {email} → candidate key ✓
- {student_id, email} → super key but NOT candidate key (not minimal)

Example: In ENROLLMENT(student_id, course_id, grade):

- {student_id, course_id} → candidate key (both attributes together needed; neither alone is sufficient)

5.3 Primary Key

Definition: The **primary key (PK)** is the candidate key selected by the database designer to serve as the principal means of uniquely identifying tuples in a relation. It is the designated unique identifier for every tuple in the table.

Formal properties:

1. **Uniqueness:** No two tuples can have the same PK value.
2. **Non-nullability:** PK attributes must **never be NULL** (entity integrity constraint).
3. **Singularity:** A table can have **only one** primary key (though it may span multiple columns — composite PK).
4. **Stability:** PK values should rarely change (they are referenced by foreign keys).
5. **Minimality:** PK should be chosen to be as small as possible.

SQL Notation:

-- Single-attribute PK

```
CREATE TABLE STUDENT (
```

```
student_id INTEGER PRIMARY KEY,
```

```

name    VARCHAR(100) NOT NULL
);

-- Composite PK
CREATE TABLE ENROLLMENT (
student_id INTEGER,
course_id  VARCHAR(10),
grade     CHAR(2),
PRIMARY KEY (student_id, course_id)
);

```

ER Notation: Attribute name is **underlined** in the ER diagram.

Example: In EMPLOYEE(emp_id, ssn, name, dept_id):

- Both **emp_id** and **ssn** could be candidate keys.
- Designer chooses **emp_id** as primary key → it gets underlined in the ER diagram.
- **ssn** becomes an **alternate key**.

Choosing a Primary Key — Best Practices:

- Prefer **surrogate keys** (system-generated integers like **AUTO_INCREMENT**) over natural keys when natural key values may change.
- Prefer **short** keys (fewer bytes = faster indexing).
- Avoid keys that encode meaning (e.g., product codes that embed category — categories change).

5.4 Alternate Key

Definition: An **alternate key** is any candidate key that was **not selected** as the primary key.

Example: In EMPLOYEE(emp_id, ssn, name):

- Both **emp_id** and **ssn** are candidate keys.
- If **emp_id** is chosen as PK, then **ssn** is an alternate key.
- Alternate keys are often enforced with **UNIQUE** constraints.

5.5 Unique Key

Definition: A **unique key** constraint ensures that all values in a column (or combination of columns) are distinct across all rows. Unlike the primary key, a table can have **multiple unique keys**, and unique keys **can contain NULL** (but at most one NULL per column).

Fundamental characteristics:

- One or more table fields can compose a unique key.
- A table may have multiple unique key columns.
- Unique key columns use **unclustered unique indexes** by default.
- A column with a unique constraint may store **one NULL** value per column.
- Unique key can serve as a **foreign key target** to maintain cross-table uniqueness.

Primary Key vs. Unique Key:

Property	Primary Key	Unique Key
NULL allowed	No	Yes (one per column)
Number per table	Exactly one	Multiple allowed
Clustered index	Yes (by default)	No (unclustered by default)
Purpose	Entity identifier	Enforce uniqueness on non-PK columns

```
CREATE TABLE EMPLOYEE (  
  emp_id    INTEGER    PRIMARY KEY,    -- PK: no NULL, one per table  
  ssn       CHAR(11)   UNIQUE,         -- Unique key: allows one NULL  
  email     VARCHAR(100) UNIQUE NOT NULL -- Unique + NOT NULL = alternate key  
  behavior  
);
```

5.6 Foreign Key

Definition: A **foreign key (FK)** is a column (or combination of columns) in a **referencing relation** whose values must match values of a **candidate key** (typically the primary key) in a **referenced relation**, or be NULL.

Purpose: Establishes and enforces a **referential link** between two tables, ensuring cross-table data consistency.

Rules:

1. FK value must exist in the referenced PK column, OR be NULL (if nullable FK).
2. The referenced column must be a **candidate key** (PK or unique key) of the referenced table.
3. FK and PK must have **compatible data types**.

SQL Syntax:

```
CREATE TABLE Purchase (  
    prodName CHAR(30) REFERENCES Product(name), -- Inline FK  
    purchDate DATETIME  
);
```

-- Or as table constraint:

```
CREATE TABLE ENROLLMENT (  
    student_id INTEGER,  
    course_id VARCHAR(10),  
    FOREIGN KEY (student_id) REFERENCES STUDENT(student_id),  
    FOREIGN KEY (course_id) REFERENCES COURSE(course_id)  
);
```

Referential Actions (on parent update/delete):

```
FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id)  
  
ON DELETE CASCADE -- Delete child rows when parent is deleted  
ON DELETE SET NULL -- Set FK to NULL when parent is deleted  
ON DELETE RESTRICT -- Prevent parent deletion if children exist  
ON DELETE NO ACTION -- Default; similar to RESTRICT but deferred  
ON UPDATE CASCADE -- Update FK when parent PK changes
```

Example:

- **PRODUCT**(name, price) — parent table
- **PURCHASE**(prodName, purchDate) — child table with FK prodName → **Product**(name)
- If we try to insert a purchase for a product that doesn't exist in **PRODUCT** → **rejected** (referential integrity).
- If we delete a product that has associated purchases → **rejected** (unless ON DELETE CASCADE).

→ **Foreign Key Constraint Diagram**

5.7 Composite Key

Definition: A **composite key** is a key that consists of **two or more attributes** that together uniquely identify a tuple. Neither attribute alone is sufficient for unique identification.

Example: In **FLIGHT_SEAT**(flight_no, seat_no, passenger_id):

- (flight_no, seat_no) → composite PK (a seat number is only unique within a flight)
- Neither flight_no alone nor seat_no alone is unique across all tuples.

```
CREATE TABLE FLIGHT_SEAT (
```

```
flight_no VARCHAR(10),
```

```
seat_no VARCHAR(5),
```

```
passenger_id INTEGER,
```

```
PRIMARY KEY (flight_no, seat_no) -- Composite PK
```

```
);
```

5.8 Keys Comparison Table

Key Type	Uniqueness	NULL Allowed	Count per Table	Minimality	Notes
Super Key	✓	—	Multiple	Not required	Any superset of a CK is a SK
Candidate Key	✓	—	Multiple	Required	Minimal super key; eligible for PK
Primary Key	✓	✗ (Never)	Exactly 1	Required	Selected CK; main identifier
Alternate Key	✓	—	Multiple	Required	Non-selected candidate key
Unique Key	✓	✓ (1 per col)	Multiple	—	Enforced via UNIQUE constraint
Foreign Key	—	✓ (if nullable)	Multiple	—	References PK/unique key of another table
Composite Key	✓ (combined)	Depends on type	—	—	Multi-attribute key of any above types

Hierarchical Relationship of Keys:

Super Key

└─ Candidate Key (minimal super key)

└─ Primary Key (one chosen CK)

└─ Alternate Key (remaining CKs)

↕

Unique Key (SQL enforcement mechanism)

Foreign Key (cross-table reference — different concept)

6. Reduction of ER Diagrams to Tables

The process of converting an ER schema into a relational schema (set of tables) is called **ER-to-Table Mapping** or **ER Schema Reduction**. The output is a set of relation schemas with clearly defined primary keys, foreign keys, and attributes.

6.1 General Rules and Algorithm

The standard algorithm (from Elmasri & Navathe, *Fundamentals of Database Systems*) involves **7 steps**:

Step 1: Map Strong Entity Sets

Step 2: Map Weak Entity Sets

Step 3: Map Binary 1:1 Relationship Sets

Step 4: Map Binary 1:N Relationship Sets

Step 5: Map Binary M:N Relationship Sets

Step 6: Map Multivalued Attributes

Step 7: Map N-ary Relationship Sets

6.2 Strong Entity Set → Table

Rule: For each **strong entity set** **E** with simple attributes $a_1, a_2, \dots, a_n$:

- Create a table **T** with columns corresponding to all simple attributes.
- The **primary key** of **T** is the primary key of **E**.
- Composite attributes → expand into their components (individual columns).
- Derived attributes → **omit** (not stored, computed at query time).

Example:

ER Entity: EMPLOYEE(emp_id [PK], first_name, last_name, dob, age [derived])

Resulting Table:

EMPLOYEE(emp_id, first_name, last_name, dob)

↑ PK [Note: age is omitted — derived attribute]

ER Entity: PROFESSOR(pid [PK], hkid, dept, rank, salary)

Resulting Table:

PROF(pid, hkid, dept, rank, salary)

↑ PK

6.3 Weak Entity Set → Table

Rule: For each **weak entity set** W with partial key pk_w, owned by strong entity E with primary key pk_e:

- Create table T with all simple attributes of W.
- Add the PK of the owner entity (pk_e) as a **foreign key** in T.
- The **primary key** of T = {pk_e, pk_w} (composite key combining owner PK + partial key).

Example:

ER: EMPLOYEE(emp_id [PK]) owns DEPENDENT(dep_name [partial key], relationship, dob)

Resulting Table:

DEPENDENT(emp_id, dep_name, relationship, dob)

↑ FK ↑ partial key

PK = (emp_id, dep_name)

FK: emp_id → EMPLOYEE(emp_id)

6.4 Composite Attributes → Columns

Rule: For composite attributes, **do not create a column for the composite attribute itself**. Instead, create **individual columns** for each of its atomic components (leaf-level sub-attributes).

Example:

- Composite attribute Name {First_Name, Last_Name} → two columns: first_name, last_name
- Composite attribute Address {Street, City, State, Zip} → four columns: street, city, state, zip

-- DO NOT do this:

-- CREATE TABLE STUDENT (student_id INTEGER, name VARCHAR, ...);

-- DO this:

```
CREATE TABLE STUDENT (  
  student_id INTEGER PRIMARY KEY,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50),  
  street VARCHAR(100),
```

```

city    VARCHAR(50),
state   CHAR(2),
zip     CHAR(10)
);

```

6.5 Multivalued Attributes → Separate Table

Rule: For each **multivalued attribute** M of entity E with PK pk_e:

- Create a **new table** T_M with:
 - A column for the multivalued attribute value.
 - A foreign key column referencing the PK of E.
- The **PK of T_M** = {pk_e, M} (to prevent duplicate values for the same entity).

Example:

- Entity STUDENT(student_id [PK], name, phone [multivalued])

Resulting Tables:

STUDENT(student_id, name) ← main table (phone omitted)

↑ PK

STUDENT_PHONE(student_id, phone) ← multivalued attribute table

↑ FK ↑

PK = (student_id, phone)

FK: student_id → STUDENT(student_id)

Example: A student with student_id = 101 and phones {9876543210, 8765432109}:

student_id	phone
101	9876543210
101	8765432109

6.6 Derived Attributes

Rule: Derived attributes are **not stored** as columns in any table. They are computed at query time using SQL expressions or application logic.

Example: Age derived from DOB:

-- At query time:

```
SELECT name, EXTRACT(YEAR FROM AGE(dob)) AS age
FROM STUDENT;
```

Exception: If performance requires it, some designers store derived attributes and maintain them via triggers — but this is a physical design decision, not a logical one.

6.7 Binary Relationships → Tables (Cardinality-based)

The mapping of relationships depends critically on the **cardinality ratio** and **participation constraints**.

Case 1: Binary M:N Relationship

Rule: Create a **new relationship table** (junction/bridge table) with:

- Foreign key to the PK of entity set A.
- Foreign key to the PK of entity set B.
- Any **descriptive attributes** of the relationship.
- PK = composite {FK_A, FK_B}.

Example: STUDENT(sid) —[M:N enrolled_in]— CLASS(cid) with relationship attributes (year, grade)

Tables Generated:

STUDENT(sid, ...)

CLASS(cid, ...)

TAKE(sid, cid, year, grade) ← relationship table

↑FK ↑FK

PK = (sid, cid)

FK: sid → STUDENT(sid)

FK: cid → CLASS(cid)

→ **M:N Relationship Table Mapping Diagram**

Case 2: Binary 1:N Relationship

Rule: No new table needed. Instead, add the PK of the "**one**" side as a **foreign key** in the table of the "**many**" side.

Example: PROF(pid) —[1:N teaches]— CLASS(cid) (one professor teaches many classes)

Tables Generated:

PROF(pid, ...)

CLASS(cid, year, pid, ...) ← pid added as FK to CLASS

↑ FK → PROF(pid)

Case 3: Binary 1:1 Relationship

Rule: Two options — choose based on participation constraints:

- **Option A (preferred when one side has total participation):** Add the PK of one entity as a FK in the other entity's table. Choose the side with **total participation** as the FK-holder (minimizes NULLs).
- **Option B:** Merge both entity sets into a single table (if they always co-exist).

Example: EMPLOYEE(eid) —[1:1 manages]— DEPARTMENT(did)

- If every employee manages at most one dept, and every dept has exactly one manager:

Option A:

EMPLOYEE(eid, name, did) ← did added as FK

FK → DEPARTMENT(did)

OR

DEPARTMENT(did, name, manager_eid) ← manager_eid added as FK

FK → EMPLOYEE(eid)

Cardinality Mapping Decision Table

Cardinality	Approach	New Table?	FK Placement
M:N	Create junction table	Yes	Junction table holds FKs to both
1:N	Migrate PK to "many" side	No	FK in the "many" entity's table
N:1	Migrate PK to "many" side	No	FK in the "many" entity's table
1:1	Migrate PK to total-participation side	No	FK in either table (prefer total participant)

6.8 Complete Worked Example

Scenario: University Database ER Schema

Entities:

- STUDENT(Std_Id [PK], Std_Name, DOB, Address{City, Street}, Hobby [multivalued])
- TEACHER(Teach_Id [PK], Teach_Name)
- SUBJECT(Sub_Id [PK], Sub_Name)
- COURSE(Crs_Id [PK], Crs_Name)

Relationships:

- STUDENT [M:N] enrolled_in COURSE
- TEACHER [1:N] teaches SUBJECT
- COURSE [1:N] consists_of SUBJECT

Step 1 — Map all Strong Entities:

STUDENT(Std_Id, Std_Name, DOB, City, Street)

↑ PK [Address is composite → City, Street become columns]

[Hobby omitted here → handled separately]

TEACHER(Teach_Id, Teach_Name)

↑ PK

SUBJECT(Sub_Id, Sub_Name)

↑ PK

COURSE(Crs_Id, Crs_Name)

↑ PK

Step 2 — Handle Multivalued Attribute (Hobby):

STUDENT_HOBBY(Std_Id, Hobby)

↑ FK ↑

PK = (Std_Id, Hobby)

FK: Std_Id → STUDENT(Std_Id)

Step 3 — Handle M:N Relationship (enrolled_in):

ENROLLMENT(Std_Id, Crs_Id, EnrollYear, Grade)

↑ FK ↑ FK

PK = (Std_Id, Crs_Id)

FK: Std_Id → STUDENT(Std_Id)

FK: Crs_Id → COURSE(Crs_Id)

Step 4 — Handle 1:N Relationship (teaches):

SUBJECT(Sub_Id, Sub_Name, Teach_Id)

↑ FK → TEACHER(Teach_Id)

[No new table; FK migrated to many-side (SUBJECT)]

Final Relational Schema:

STUDENT(Std_Id, Std_Name, DOB, City, Street)

PK: Std_Id

TEACHER(Teach_Id, Teach_Name)

PK: Teach_Id

SUBJECT(Sub_Id, Sub_Name, Teach_Id)

PK: Sub_Id

FK: Teach_Id → TEACHER(Teach_Id)

COURSE(Crs_Id, Crs_Name)

PK: Crs_Id

STUDENT_HOBBY(Std_Id, Hobby)

PK: (Std_Id, Hobby)

FK: Std_Id → STUDENT(Std_Id)

ENROLLMENT(Std_Id, Crs_Id, EnrollYear, Grade)

PK: (Std_Id, Crs_Id)

FK: Std_Id → STUDENT(Std_Id)

FK: Crs_Id → COURSE(Crs_Id)

SQL DDL for the above schema:

```
CREATE TABLE STUDENT (  
Std_Id    INTEGER    PRIMARY KEY,  
Std_Name  VARCHAR(100) NOT NULL,  
DOB       DATE,  
City      VARCHAR(50),  
Street    VARCHAR(100)  
);
```

```
CREATE TABLE TEACHER (  
Teach_Id  INTEGER    PRIMARY KEY,  
Teach_Name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE SUBJECT (  
Sub_Id    INTEGER    PRIMARY KEY,  
Sub_Name  VARCHAR(100) NOT NULL,  
Teach_Id  INTEGER,  
FOREIGN KEY (Teach_Id) REFERENCES TEACHER(Teach_Id)  
ON DELETE SET NULL
```

);

```
CREATE TABLE COURSE (  
  Crs_Id    INTEGER    PRIMARY KEY,  
  Crs_Name  VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE STUDENT_HOBBY (  
  Std_Id    INTEGER,  
  Hobby     VARCHAR(50),  
  PRIMARY KEY (Std_Id, Hobby),  
  FOREIGN KEY (Std_Id) REFERENCES STUDENT(Std_Id)  
    ON DELETE CASCADE  
);
```

```
CREATE TABLE ENROLLMENT (  
  Std_Id    INTEGER,  
  Crs_Id    INTEGER,  
  EnrollYear INTEGER,  
  Grade     CHAR(2),  
  PRIMARY KEY (Std_Id, Crs_Id),  
  FOREIGN KEY (Std_Id) REFERENCES STUDENT(Std_Id)  
    ON DELETE CASCADE,  
  FOREIGN KEY (Crs_Id) REFERENCES COURSE(Crs_Id)  
    ON DELETE CASCADE  
);
```

7. Quick Revision Cheatsheet

ER Model — Core Concepts

Concept	Symbol	Key Property
Strong Entity	Single rectangle	Has its own PK
Weak Entity	Double rectangle	Depends on owner entity; uses partial key
Simple Attribute	Single oval	Atomic, indivisible
Composite Attribute	Oval with sub-ovals	Decomposed into components in table
Multivalued Attribute	Double oval	Gets own separate table
Derived Attribute	Dashed oval	Not stored; computed at runtime
Key Attribute	Underlined oval	Unique identifier of entity
Relationship	Diamond	Association between entities
Identifying Relationship	Double diamond	Connects weak entity to owner
Total Participation	Double line	Entity must participate in relationship
Partial Participation	Single line	Entity may or may not participate

Cardinality — Quick Reference

Type	Meaning	FK Placement Strategy
1:1	One—one	FK in total-participation table
1:N	One—many	FK in the "many" side table
M:N	Many—many	New junction table with composite PK

Integrity Constraints — Summary

Constraint	Scope	Core Rule
Domain	Single attribute	Value must be in the attribute's domain
Entity Integrity	PK	PK attribute(s) can never be NULL
Referential Integrity	FK	FK must match an existing PK or be NULL
Key	Relation	No two tuples may have the same key value

ER to Table — Decision Algorithm

For each ENTITY SET:

- If STRONG: Create table with all simple attributes; expand composites
- If WEAK: Create table; PK = (owner_PK + partial_key); add FK to owner

For each ATTRIBUTE:

- SIMPLE: → Column in entity table
- COMPOSITE: → Expand into component columns
- MULTIVALUED: → New table (entity_PK + attribute); PK = both
- DERIVED: → Omit from schema

For each RELATIONSHIP:

- M:N: → New junction table; PK = (FK1, FK2); include descriptive attrs
- 1:N: → Add PK of "1-side" as FK in "N-side" table
- 1:1: → Add FK to total-participation side; OR merge tables

Keys — One-Line Definitions

Key	One-Line Definition
Super Key	Any set of attributes that uniquely identifies a tuple (may have extras)
Candidate Key	Minimal super key — no redundant attributes
Primary Key	The chosen candidate key; no NULLs allowed
Alternate Key	Candidate key not chosen as PK
Unique Key	SQL-enforced uniqueness; allows one NULL per column
Foreign Key	Attribute(s) referencing another table's PK/unique key
Composite Key	Key spanning multiple attributes

Notes compiled for RV University DBMS Unit 2 — CIE 1 Examination. Scope: ER Model Concepts → ER Notation → Mapping Constraints → Integrity Constraints → Keys → ER to Table Reduction. References: Elmasri & Navathe "Fundamentals of Database Systems", Silberschatz "Database System Concepts", course slides (DBMS Team, RV University).